



Profiling Django views for SQL queries

>>Optimizing Django code by profiling the views for SQL queries> by Shubham Jain and Vivek Prakash

We at HackerEarth regularly conduct 24-hours internal hackathons usually once a month to boost ourselves to get familiar with new technologies and to come up with great ideas and hacks to increase our productivity. A hackathon project can be anything from creating a new product to creating some tools which helps our own development. In the hackathon in Dec 2015, we came up with the idea to create a profiler which could tell which code inside Django views causes SQL queries so that we can optimize them easily.

Initial thoughts

There are already many good Django packages to profile views for SQL queries. One of them is django-toolbar which we already use. Django-toolbar is great but it shows all raw SQL queries which are happening inside a view and you have to analyze each query, see the whole trace back and figure out which line of code inside the view is triggering the query. This way, you can only figure out the line number of the code in a file, not the exact function or expression or attribute access which is causing the query. What we wanted was that a profiler should tell about the exact expressions which trigger the SQL queries.

There were some solutions which came up in our mind to get it done like using trace backs or by manipulating AST of the Python function. Python's AST module can parse and modify code during runtime and we were fascinated about using it in the future.

What is AST?

AST stands for Abstract Syntax Tree. Python converts all Python code to AST as one of the compiling step when creating its bytecode. Same functionality it provides dynamically through AST module. We can convert any Python code to AST at runtime using this library which we can compile to bytecode and execute the bytecode too.

Here is an example. Let's have the code of a function which returns the square of a number

```
code = """def f(x):  
    s = x * x  
    return s  
"""
```

We can convert this code to AST using ast.parse method

```
import ast  
module_ast = ast.parse(code)  
func_ast = module_ast.body[0]
```

*Coding Matters



*Android Studio 2.2

>>Latest version of Google's Android development studio released with improved C++ support and more.

<http://dgit.in/AS2dot2>



*Oracle beats Amazon

>>New Oracle Database-as-a-Service is 20 years ahead of Amazon Web Services.

<http://dgit.in/OrclvsAmzn>



*Vim8.0, Emacs 25.1 out

>>First major Vim release in 10 years and Emacs 25.1 with dynamic library support released.

<http://dgit.in/VmEmcs>